

Task Overview

Task Overview

The goal of this task was to implement a centralized monitoring solution for the infrastructure and application servers using Grafana Alloy, Prometheus, Loki, and Grafana.

The setup aimed to:

- Collect system and container metrics from all servers.
- Aggregate application logs from containers and the system journal.
- Send logs and metrics to Loki and Prometheus endpoints for visualization in Grafana.
- Extend monitoring by integrating cAdvisor, Redis exporter, and Valkey exporter for container and database insights.
- Introduce persistent storage for data retention and durability.
- Enable richer log collection through Docker socket integration.

The implementation started in the development environment and was later extended to the Test and production servers.

Devops Apprentice: Akintola Oyinkansola

Contents

Contents

1. Procedures
2. Issue(s) Encountered
3. Recommendations
4. Wrap-up

Project Architecture and Components

The setup included the following key components:

- Grafana Alloy: Agent responsible for collecting metrics and logs, and forwarding them to Loki and Prometheus.
- Prometheus: Time-series database for storing and querying metrics.
- Loki: Log aggregation system.
- Grafana: Visualization layer for both metrics and logs.

Alloy was configured to forward:

- Metrics → Prometheus endpoint (<http://20.224.239.134:9091/api/v1/write>)
- Logs → Loki endpoint 1 <http://<ipaddress1>:3100/loki/api/v1/push>)
- Logs → Loki endpoint 2 (<http://<ipaddress2>:3100/loki/api/v1/push>)

Procedures

Procedures

PART 1

Step 1: Repository Cloning and Directory Setup

- Clone the official Grafana Alloy Scenarios repository:
 - `git clone https://github.com/grafana/alloy-scenarios.git`
- Navigate into the Linux scenario directory (only this part was used):
 - `cd alloy-scenarios/linux`
- A parent directory named monitoring was created on the server to organize all configuration files:
 - `mkdir -p ~/monitoring/alloy-scenarios/linux`
 - `cd ~/monitoring/alloy-scenarios/linux`

This structure ensured all monitoring-related files including the `confi.alloy`, `Docker-compose.yml`, `loki-config.yaml`, `prom-config.yaml` setup were contained in one place.

Step 2: Configuring Alloy

This step outlines how Alloy was configured to collect, process, and forward observability data including system metrics from `node_exporter`, logs from the system journal, and application logs to Loki.

Base Configuration (Infrastructure Server)

The initial setup was performed on the infrastructure server, where Alloy acted as the observability agent, collecting metrics and logs and forwarding them to local instances of Prometheus and Loki deployed via Docker Compose.

This configuration performs the following:

- **Metrics Collection:**
 - Collects system-level metrics (CPU, memory, filesystem, and network) through the embedded `node_exporter`.
- **Log Collection:**
 - Reads logs from `/var/log/` and the systemd journal, labeling them for easy querying in Grafana.
- **Forwarding Data:**
 - Metrics → Local Prometheus (<http://prometheus:9090/api/v1/write>)
 - Logs → Local Loki (`http://loki:3100/loki/api/v1/push`)
- **Reliability:**
 - Live debugging is enabled for troubleshooting Alloy pipeline behavior.

- **config.alloy**

// This block relabels metrics coming from `node_exporter` to add standard labels


```

discovery.relabel "integrations_node_exporter" {
  targets = prometheus.exporter.unix.integrations_node_exporter.targets

  rule {
    // Set the instance label to the hostname of the machine
    target_label = "instance"
    replacement = constants.hostname
  }

  rule {
    // Set a standard job name for all node_exporter metrics
    target_label = "job"
    replacement = "integrations/node_exporter"
  }
}

// Configure the node_exporter integration to collect system metrics
prometheus.exporter.unix "integrations_node_exporter" {
  // Disable unnecessary collectors to reduce overhead
  disable_collectors = ["ipvs", "btrfs", "infiniband", "xfs", "zfs"]
  enable_collectors = ["meminfo"]

  filesystem {
    // Exclude filesystem types that aren't relevant for monitoring
    fs_types_exclude =
    "^(autofs|binfmt_misc|bpf|cgroup2?|configfs|debugfs|devpts|devtmpfs|tmpfs|fusectl|hugetlbfs|
    iso9660|mqueue|nsfs|overlay|proc|procfs|pstore|rpc_pipefs|securityfs|selinuxfs|squashfs|sysfs|
    tracefs)$"
    // Exclude mount points that aren't relevant for monitoring
    mount_points_exclude = "^(dev|proc|run/credentials/.+|sys|var/lib/docker/.+)($/)"
    // Timeout for filesystem operations
    mount_timeout = "5s"
  }

  netclass {
    // Ignore virtual and container network interfaces
    ignored_devices = "^(veth.*|cali.*[a-f0-9]{15})$"
  }

  netdev {
    // Exclude virtual and container network interfaces from device metrics
    device_exclude = "^(veth.*|cali.*[a-f0-9]{15})$"
  }
}

// Define how to scrape metrics from the node_exporter

```

```

prometheus.scrape "integrations_node_exporter" {
  scrape_interval = "15s"
  // Use the targets with labels from the discovery.relabel component
  targets = discovery.relabel.integrations_node_exporter.output
  // Send the scraped metrics to the relabeling component
  forward_to = [prometheus.remote_write.local.receiver]
}

// Define where to send the metrics for storage
prometheus.remote_write "local" {
  endpoint {
    // Send metrics to a locally running Prometheus instance
    url = "http://prometheus:9090/api/v1/write"
  }
}

// Collect logs from systemd journal for node_exporter integration
loki.source.journal "logs_integrations_integrations_node_exporter_journal_scrape" {
  // Only collect logs from the last 24 hours
  max_age = "24h0m0s"
  // Apply relabeling rules to the logs
  relabel_rules =
discovery.relabel.logs_integrations_integrations_node_exporter_journal_scrape.rules
  // Send logs to the local Loki instance
  forward_to = [loki.write.local.receiver]
}

// Define which log files to collect for node_exporter
local.file_match "logs_integrations_integrations_node_exporter_direct_scrape" {
  path_targets = [{
    // Target localhost for log collection
    __address__ = "localhost",
    // Collect standard system logs
    __path__ = "/var/log/{syslog,messages,*.log}",
    // Add instance label with hostname
    instance = constants.hostname,
    // Add job label for logs
    job = "integrations/node_exporter",
  }]
}

// Define relabeling rules for systemd journal logs
discovery.relabel "logs_integrations_integrations_node_exporter_journal_scrape" {
  targets = []

  rule {
    // Extract systemd unit information into a label

```

```

    source_labels = ["__journal__systemd_unit"]
    target_label = "unit"
}

rule {
    // Extract boot ID information into a label
    source_labels = ["__journal__boot_id"]
    target_label = "boot_id"
}

rule {
    // Extract transport information into a label
    source_labels = ["__journal__transport"]
    target_label = "transport"
}

rule {
    // Extract log priority into a level label
    source_labels = ["__journal_priority_keyword"]
    target_label = "level"
}
}

// Collect logs from files for node_exporter
loki.source.file "logs_integrations_integrations_node_exporter_direct_scrape" {
    // Use targets defined in local.file_match
    targets =
local.file_match.logs_integrations_integrations_node_exporter_direct_scrape.targets
    // Send logs to the local Loki instance
    forward_to = [loki.write.local.receiver]
}

// Define where to send logs for storage
loki.write "local" {
    endpoint {
        // Send logs to a locally running Loki instance
        url = "http://loki:3100/loki/api/v1/push"
    }
}

// Enable live debugging features (empty config means use defaults)
livedebugging{}

```

Step 3: Docker Compose Configuration

- Docker Compose file: The docker-compose.yml file defines all services in the monitoring stack, Loki, Prometheus, Grafana, and Alloy.

docker-compose.yml

```
version: '3.8'

services:

  loki:

    image: grafana/loki:${GRAFANA_LOKI_VERSION:-3.5.2}

    ports:

      - 3100:3100/tcp

    volumes:

      - ./loki-config.yaml:/etc/loki/local-config.yaml

      - loki-data:/loki

    command: -config.file=/etc/loki/local-config.yaml

    mem_limit: 512m

    cpus: 0.5

    restart: unless-stopped

  prometheus:

    image: prom/prometheus:${PROMETHEUS_VERSION:-v3.5.0}

    command:

      - --web.enable-remote-write-receiver

      - --config.file=/etc/prometheus/prometheus.yml
```

```
ports:

  - 9091:9090/tcp

volumes:

  - ./prom-config.yaml:/etc/prometheus/prometheus.yml

  - prometheus-data:/prometheus

mem_limit: 1g

cpus: 1.0

restart: unless-stopped


grafana:

  image: grafana/grafana:${GRAFANA_VERSION:-12.0.2}

  environment:

    - GF_AUTH_ANONYMOUS_ORG_ROLE=Admin

    - GF_AUTH_ANONYMOUS_ENABLED=true

    - GF_AUTH_BASIC_ENABLED=false

  ports:

    - 3000:3000/tcp

  volumes:

    - grafana-data:/var/lib/grafana

  entrypoint:

    - sh

    - -euc

    - |

      mkdir -p /etc/grafana/provisioning/datasources

      cat <<EOF > /etc/grafana/provisioning/datasources/ds.yaml
```

```
    apiVersion: 1

    datasources:

    - name: Loki

      type: loki

      access: proxy

      orgId: 1

      url: http://loki:3100

      basicAuth: false

      isDefault: false

      version: 1

      editable: false

    - name: Prometheus

      type: prometheus

      orgId: 1

      url: http://prometheus:9090

      basicAuth: false

      isDefault: true

      version: 1

      editable: false

    EOF

    /run.sh

  mem_limit: 512m

  cpus: 0.5

  restart: unless-stopped
```

```

alloy:

  image: grafana/alloy:${GRAFANA_ALLOY_VERSION:-v1.10.0}

  volumes:

    - ./config.alloy:/etc/alloy/config.alloy

    - /var/log:/var/log:ro

    - /run/log/journal:/run/log/journal:ro

    - /etc/machine-id:/etc/machine-id:ro

  command: run --server.http.listen-addr=0.0.0.0:12345
--storage.path=/var/lib/alloy/data /etc/alloy/config.alloy

  mem_limit: 256m

  cpus: 0.25

  restart: unless-stopped

volumes:

  grafana-data:

  prometheus-data:

  loki-data:

```

- **Loki Configuration with Persistent Storage:** This configuration ensures that Loki stores data in `/tmp/loki/chunks` and uses the `loki-data` Docker volume for persistent storage, allowing logs to be retained across container restarts.
- **loki-config.yaml**

```

# This is a complete configuration to deploy Loki backed by the
filesystem.

# The index will be shipped to the storage via tsdb-shipper.

```

```
auth_enabled: false

limits_config:

  allow_structured_metadata: true

  volume_enabled: true
```

```
server:

  http_listen_port: 3100
```

```
common:

  ring:

    instance_addr: 0.0.0.0

    kvstore:

      store: inmemory

  replication_factor: 1

  path_prefix: /tmp/loki
```

```
schema_config:

  configs:

    - from: 2020-05-15

      store: tsdb

      object_store: filesystem

      schema: v13

    index:

      prefix: index_
```



```

    period: 24h

storage_config:

  tsdb_shipper:

    active_index_directory: /tmp/loki/index

    cache_location: /tmp/loki/index_cache

  filesystem:

    directory: /tmp/loki/chunks

pattern_ingester:

  enabled: true

# Note: We are setting the max chunk age far lower than the default
# expected value

# This is due to the fact this scenario is used within the LogCLI demo
# and we need a short flush time.

# To show how logcli stats --since 24h '{service_name="Delivery World",
# package_size="Large"}' works.

ingester:

  max_chunk_age: 5m # Should be 2 hours

```

- Prometheus Configuration: This configuration allows Prometheus to scrape metrics exposed by the Alloy service at port 12345.

- **prom-config.yaml**

```

# my global config

global:

```

```

scrape_interval: 15s

evaluation_interval: 15s

scrape_configs:
  - job_name: "alloy"

    static_configs:
      - targets: ["alloy:12345"]

```

Step 4: Deploy and Start All Services

- Once all configuration files are in place, deploy the monitoring stack using Docker Compose:
 - `docker-compose up -d`

Step 5: Verify Deployment

- Run the following commands to confirm that all containers are running:
 - `docker ps`

```

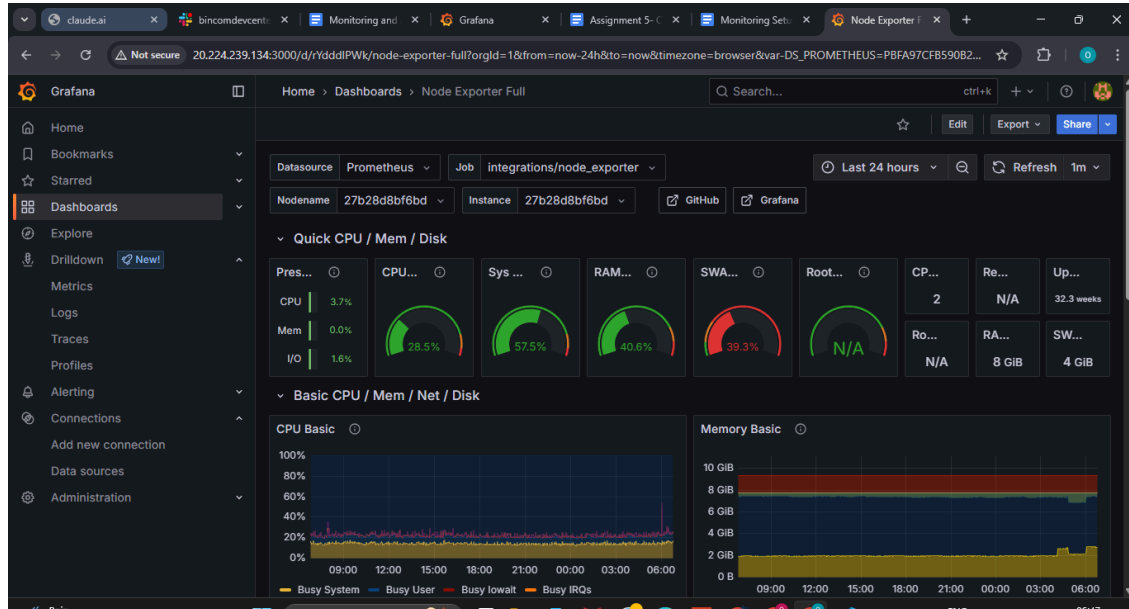
oyin_bincom@infrastructure-vm:~/monitoring$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
5b690db659eb   grafana/grafana:12.0.2             "sh -euc 'mkdir -p /_..." 7 days ago    Up 7 days    0.0.0.0:3000->3000/tcp
, ::3000->3000/tcp
27b28d8bf6bd   grafana/alloy:v1.10.0              "/bin/alloy run --se..." 7 days ago    Up 7 days
monitoring-grafana-1
fd9c4ef3adb1   prom/prometheus:v3.5.0             "/bin/prometheus --w..." 7 days ago    Up 7 days    0.0.0.0:9091->9090/tcp
, ::9091->9090/tcp
1e3d76d659dc   grafana/loki:3.5.2                 "/usr/bin/loki -conf..." 7 days ago    Up 7 days    0.0.0.0:3100->3100/tcp
, ::3100->3100/tcp
d18775435314   phpmyadmin/phpmyadmin              "/docker-entrypoint...." 6 weeks ago    Up 6 weeks    0.0.0.0:8086->80/tcp,
, ::8086->80/tcp
ca2601658cca   wordpress:latest                   "docker-entrypoint.s..." 6 weeks ago    Up 6 weeks    0.0.0.0:8084->80/tcp,
, ::8084->80/tcp
32820f2fa675   mysql:5.7                           "docker-entrypoint.s..." 6 weeks ago    Up 6 weeks    3306/tcp, 33060/tcp
mysql
f8e4e91cce07   odoo:17.0                           "/entrypoint.sh odoo..." 6 weeks ago    Up 6 weeks    0.0.0.0:8069->8069/tcp
, ::8069->8069/tcp, 8071-8072/tcp
c3667392dfec   postgres:15                         "docker-entrypoint.s..." 6 weeks ago    Up 6 weeks    5432/tcp
odoo-docker-postgres-1
18ef6884343c   mysql:latest                       "docker-entrypoint.s..." 2 months ago    Up 2 months    33060/tcp, 0.0.0.0:808
2->3306/tcp, ::8082->3306/tcp
03b17c520f08   wordpress:latest                   "docker-entrypoint.s..." 2 months ago    Up 2 months    0.0.0.0:8083->80/tcp,
, ::8083->80/tcp
ed72c68b5d87   grafana/promtail:2.9.2             "/usr/bin/promtail -..." 6 months ago    Up 6 months
promtail
e678a74d7750   mysql:5.7                           "docker-entrypoint.s..." 7 months ago    Up 7 months    33060/tcp, 0.0.0.0:808
1->3306/tcp, ::8081->3306/tcp
7e876e28c96a   prom/node-exporter                 "/bin/node_exporter ..." 7 months ago    Up 7 months    0.0.0.0:8087->9100/tcp
, ::8087->9100/tcp
node-exporter

```

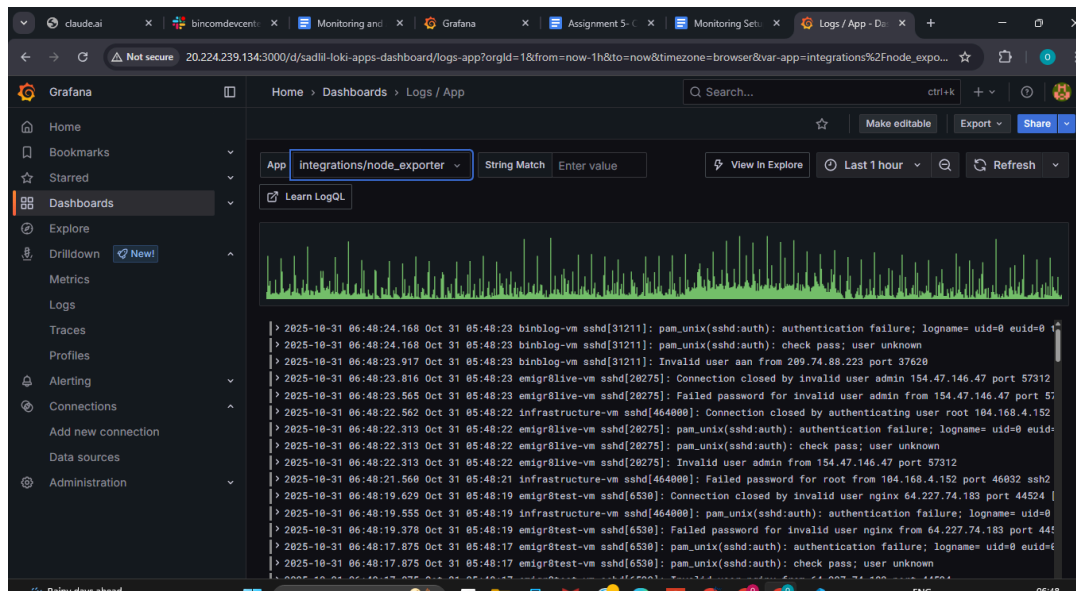
You should see four containers: grafana, prometheus, loki, and alloy.

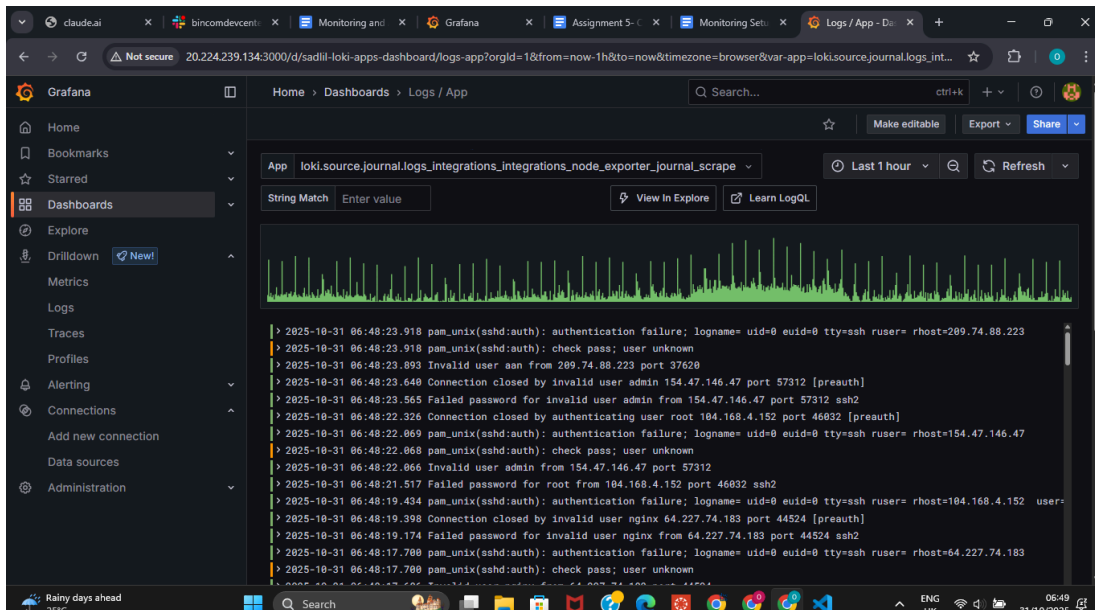
Step 6: Access the Grafana Dashboard

- Open your browser and navigate to:
 - Grafana UI: `http://<server-ip>:3000` (default credentials: admin / admin)
- Configure Prometheus and Loki as Grafana data sources.
- Create dashboards to visualize:
 - Node metrics (CPU, memory, disk)



- Logs from Loki (system and application logs)





Part 2

This section covers the process of extending the Alloy configuration on the Test server to include additional exporters and enable container log collection.

The setup enhances observability by integrating cAdvisor, Valkey, and Redis Exporter for metrics, and by using Docker socket integration to collect logs from all running containers.

All related files including the Docker Compose definition and the Alloy configuration file are maintained under the monitoring directory on the server.

Step 7: Extending Alloy Configuration on Test Server

- Directory Setup: Create a dedicated directory for monitoring to organize your Docker Compose files and Alloy configuration:
 - `mkdir -p ~/monitoring`
 - `cd ~/monitoring`
- Inside ~/monitoring, you will have:
 - `docker-compose.yml` → to run Alloy and its monitoring dependencies
 - `config.alloy` → Alloy configuration file

Step 8: Update the Configuration File to Collect Container Metrics

Update the `config.alloy` file to start collecting container-level metrics and logs

At this stage:

- Add integrations for Redis and Valkey exporters.
- Include cAdvisor if required, to capture host and container resource metrics.
- Extend the log section to allow Alloy to ingest logs from Docker containers using the Docker socket mount.

This update ensures Alloy serves as a single observability agent, scraping metrics from exporters and forwarding container logs.

- **config.alloy**

```
// This block relabels metrics coming from node_exporter to add standard labels
discovery.relabel "integrations_node_exporter" {
  targets = prometheus.exporter.unix.integrations_node_exporter.targets

  rule {
    // Set the instance label to the hostname of the machine
    target_label = "instance"
    replacement = constants.hostname
  }

  rule {
    // Set a standard job name for all node_exporter metrics
    target_label = "job"
    replacement = "integrations/node_exporter"
  }
}

// Configure the node_exporter integration to collect system metrics
prometheus.exporter.unix "integrations_node_exporter" {
  // Disable unnecessary collectors to reduce overhead
  disable_collectors = ["ipvs", "btrfs", "infiniband", "xfs", "zfs"]
  enable_collectors = ["meminfo"]

  filesystem {
    // Exclude filesystem types that aren't relevant for monitoring
    fs_types_exclude =
    "^(autofs|binfmt_misc|bpf|cgroup2?|configfs|debugfs|devpts|devtmpfs|tmpfs|fusectl|hugetlbfs|
    iso9660|mqueue|nsfs|overlay|proc|procfs|pstore|rpc_pipefs|securityfs|selinuxfs|squashfs|sysfs|tracefs)$"
    // Exclude mount points that aren't relevant for monitoring
    mount_points_exclude = "^(/dev|/proc|/run|/credentials/.+|/sys|/var/lib/docker/.+)(/|$)"
    // Timeout for filesystem operations
    mount_timeout = "5s"
  }

  netclass {
    // Ignore virtual and container network interfaces
    ignored_devices = "^(veth.*|cali.*[a-f0-9]{15})$"
  }

  netdev {
    // Exclude virtual and container network interfaces from device metrics
    device_exclude = "^(veth.*|cali.*[a-f0-9]{15})$"
  }
}
```

```

}
}

// Define how to scrape metrics from the node_exporter
prometheus.scrape "integrations_node_exporter" {
    scrape_interval = "15s"
    // Use the targets with labels from the discovery.relabel component
    targets = discovery.relabel.integrations_node_exporter.output
    // Send the scraped metrics to the relabeling component
    forward_to = [prometheus.remote_write.local.receiver]
}

// Define where to send the metrics for storage
prometheus.remote_write "local" {
    endpoint {
        // Send metrics to a locally running Prometheus instance
        url = "http://<ipaddress1>:9091/api/v1/write"
    }
}

// NEW: cAdvisor (container metrics)
prometheus.scrape "cadvisor" {
    scrape_interval = "15s"
    targets = [{
        __address__ = "localhost:8086",
    }]
    forward_to = [prometheus.remote_write.local.receiver]
}

// NEW: Redis + Valkey Metrics
prometheus.scrape "redis_metrics" {
    scrape_interval = "15s"
    targets = [
        { __address__ = "localhost:9121" },
        { __address__ = "localhost:9122" },
    ]
    forward_to = [prometheus.remote_write.local.receiver]
}

// Process component to fan out logs to multiple Loki instances
loki.process "logs" {
    forward_to = [
        loki.write.local.receiver,
        loki.write.secondary.receiver,
    ]

    stage.static_labels {
        values = {}
    }
}

```

```
}  
}
```

```
// Collect logs from systemd journal for node_exporter integration  
loki.source.journal "logs_integrations_integrations_node_exporter_journal_scrape" {  
    // Only collect logs from the last 24 hours  
    max_age      = "24h0m0s"  
    // Apply relabeling rules to the logs  
    relabel_rules =  
discovery.relabel.logs_integrations_integrations_node_exporter_journal_scrape.rules  
    // Send logs to the process component  
    forward_to = [  
        loki.process.logs.receiver,  
    ]  
}
```

```
// Define which log files to collect for node_exporter  
local.file_match "logs_integrations_integrations_node_exporter_direct_scrape" {  
    path_targets = [{  
        // Target localhost for log collection  
        __address__ = "localhost",  
        // Collect standard system logs  
        __path__    = "/var/log/{syslog,messages,*.log}",  
        // Add instance label with hostname  
        instance    = constants.hostname,  
        // Add job label for logs  
        job         = "integrations/node_exporter",  
    }]  
}
```

```
// Define relabeling rules for systemd journal logs  
discovery.relabel "logs_integrations_integrations_node_exporter_journal_scrape" {  
    targets = []
```

```
    rule {  
        // Extract systemd unit information into a label  
        source_labels = ["__journal__systemd_unit"]  
        target_label  = "unit"  
    }
```

```
    rule {  
        // Extract boot ID information into a label  
        source_labels = ["__journal__boot_id"]  
        target_label  = "boot_id"  
    }
```

```
    rule {  
        // Extract transport information into a label
```

```

    source_labels = ["__journal__transport"]
    target_label = "transport"
}

rule {
    // Extract log priority into a level label
    source_labels = ["__journal_priority_keyword"]
    target_label = "level"
}
}

// Collect logs from files for node_exporter
loki.source.file "logs_integrations_integrations_node_exporter_direct_scrape" {
    // Use targets defined in local.file_match
    targets =
local.file_match.logs_integrations_integrations_node_exporter_direct_scrape.targets
    // Send logs to the process component
    forward_to = [
        loki.process.logs.receiver,
    ]
}

// Discover all Docker containers
discovery.docker "containers" {
    host = "unix:///var/run/docker.sock"
}

// Filter and relabel for specific containers
discovery.relabel "docker_logs" {
    targets = discovery.docker.containers.targets

    // Exclude Alloy itself and exporters (not useful logs)
    rule {
        source_labels = ["__meta_docker_container_name"]
        regex          = ".*(alloy|exporter).*"
        action         = "drop"
    }

    // Set container name as a label
    rule {
        source_labels = ["__meta_docker_container_name"]
        regex          = "/(.*)"
        target_label   = "container"
    }

    // Set image name as a label
    rule {
        source_labels = ["__meta_docker_container_image"]

```



```

    target_label = "image"
}

// Add hostname
rule {
    target_label = "instance"
    replacement = constants.hostname
}

// Add job label
rule {
    target_label = "job"
    replacement = "docker/containers"
}
}

// Collect logs from Docker containers
loki.source.docker "docker_logs" {
    host      = "unix:///var/run/docker.sock"
    targets   = discovery.relabel.docker_logs.output
    forward_to = [loki.process.logs.receiver]

    refresh_interval = "30s"
}

// Define where to send logs for storage
loki.write "local" {
    endpoint {
        // Send logs to a locally running Loki instance
        url = "http://<ipaddress1>:3100/loki/api/v1/push"
    }
}

// Define where to send logs for storage (second Loki)
loki.write "secondary" {
    endpoint {
        url = "http://<ipaddress2>:3100/loki/api/v1/push"
    }
}

// Enable live debugging features (empty config means use defaults)
livedebugging {
}

```

Step 9: Define the Docker Compose File

Create the docker-compose.yml file under ~/monitoring.

This file defines Alloy and the two exporters, each configured to restart automatically and expose metrics over the host network.

- Ensure Alloy can read container logs by using the mounted Docker socket:
 - - /var/run/docker.sock:/var/run/docker.sock:ro

This allows Alloy to collect and process logs from running containers for centralized visibility.

- **docker-compose.yml**

```
version: '3.8'

services:
  alloy:
    image: grafana/alloy:latest
    container_name: alloy
    restart: unless-stopped
    network_mode: host
    privileged: true
    command:
      - run
      - --server.http.listen-addr=0.0.0.0:12345
      - --storage.path=/var/lib/alloy
      - /etc/alloy/config.alloy
    volumes:
      - ./config.alloy:/etc/alloy/config.alloy:ro
      - /var/log:/var/log:ro
      - /var/lib/alloy:/var/lib/alloy
      - /var/run/docker.sock:/var/run/docker.sock:ro
    environment:
      - HOSTNAME=${HOSTNAME}
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

# Optional: Redis Exporter (if you want this managed by compose too)
redis_exporter:
  image: oliver006/redis_exporter:latest
  container_name: redis_exporter
  restart: unless-stopped
  network_mode: host
  command:
    - --redis.addr=127.0.0.1:7654
```

```

    - --redis.password=MmYROWyHpokulqGGOxaBXyC9EVuj
logging:
  driver: "json-file"
  options:
    max-size: "10m"
    max-file: "3"

# Optional: Valkey Exporter
valkey_exporter:
  image: oliver006/redis_exporter:latest
  container_name: valkey_exporter
  restart: unless-stopped
  network_mode: host
  command:
    - --redis.addr=127.0.0.1:8087
    - --web.listen-address=:9122
  logging:
    driver: "json-file"
    options:
      max-size: "10m"
      max-file: "3"

```

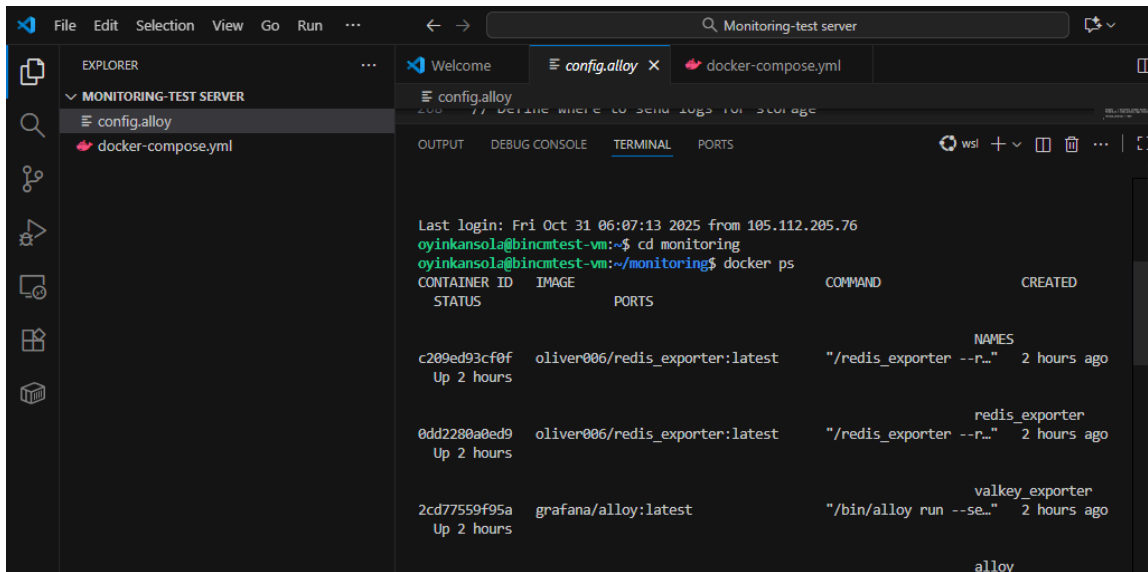
Step 10: Deploy the Monitoring Stack

- Run the stack:
 - `docker-compose up -d`

This starts Alloy, Redis Exporter, and Valkey Exporter simultaneously.

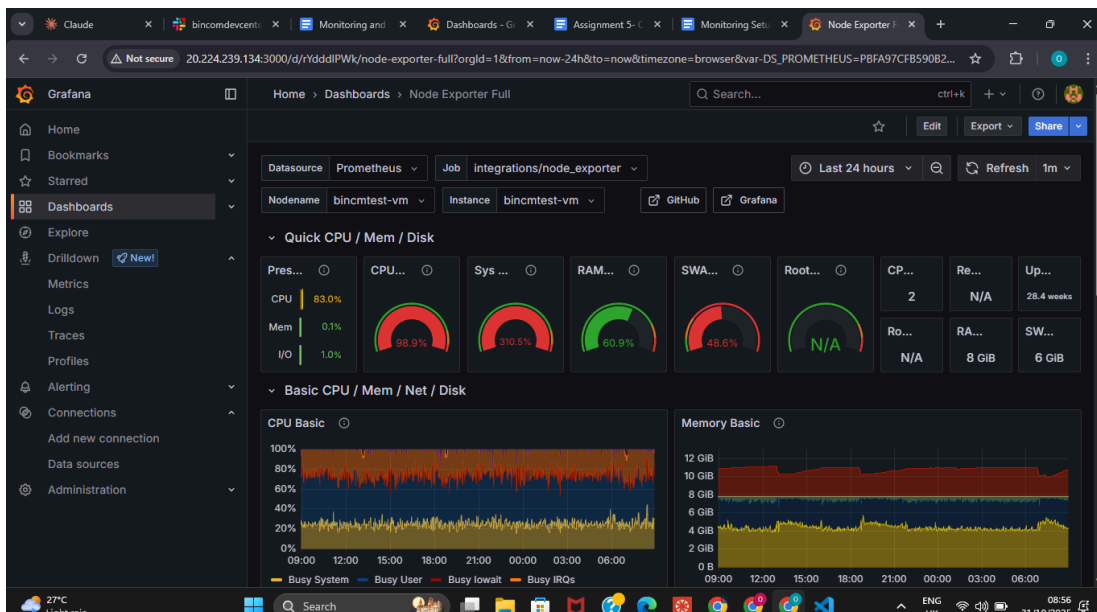
Because the exporters are defined within the same compose file, Alloy automatically discovers and scrapes them using the targets specified in `config.alloy`.

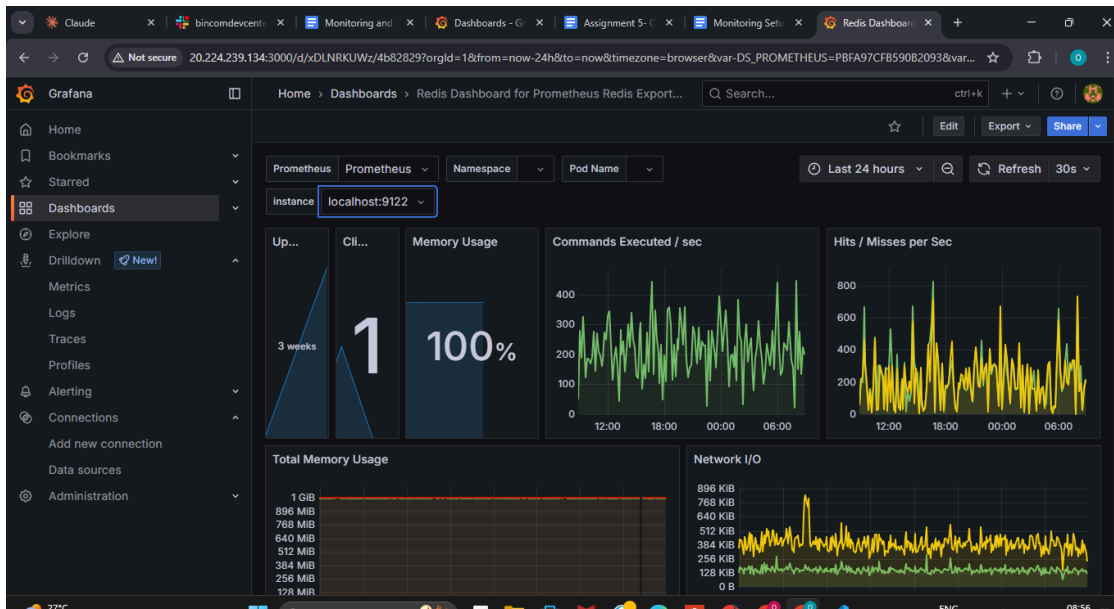
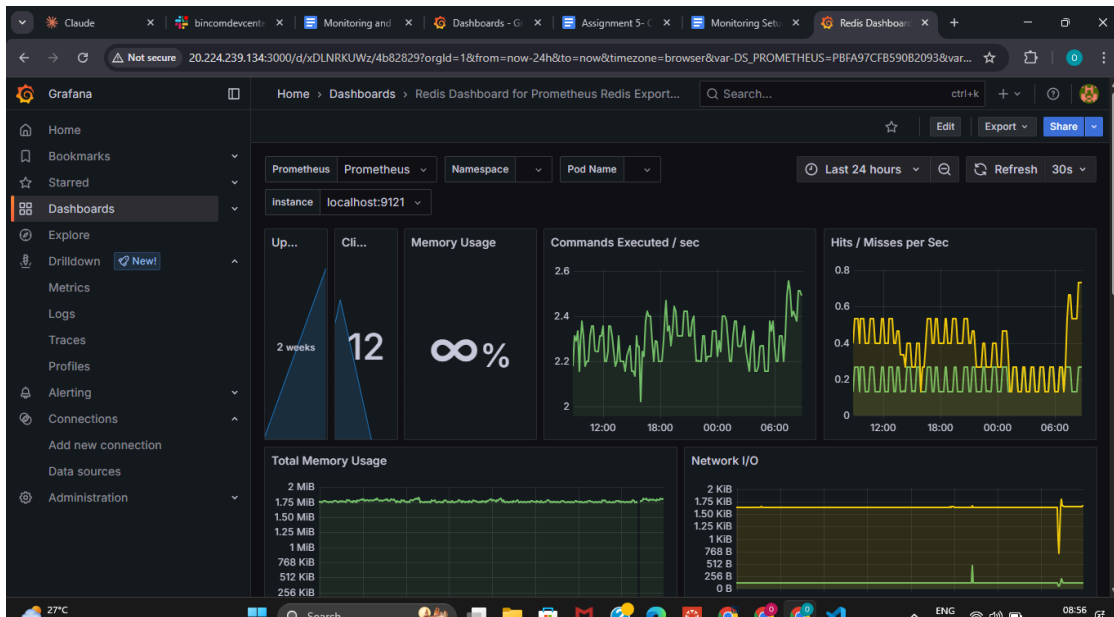
- Verify Setup and check running containers:
 - `docker ps`
- Expected containers:
 - Alloy
 - Redis_exporter
 - valkey_exporter



Step 11: Validate Monitoring Outputs

- Check the Alloy dashboard to confirm:
 - Container metrics are being collected from Node_Exporter, cAdvisor, Valkey, and Redis Exporter.





- Container logs are visible from both loki endpoints, confirming the Docker socket integration is functioning properly.

Grafana interface showing logs for the 'aleks_wordpress' container. The left sidebar contains navigation options: Home, Bookmarks, Starred, Dashboards, Explore, Drilldown, Metrics, Logs, Traces, Profiles, Alerting, Connections, Add new connection, Data sources, and Administration. The main panel displays a list of log entries for the 'aleks_wordpress' container, including timestamps, log levels, and messages. A search bar at the top allows filtering by 'container' and 'String Match'. The right sidebar shows a detailed view of the selected log entry, including the full log message and associated metadata.

Grafana interface showing logs for the 'sociallenderblogDev' container. The left sidebar contains navigation options: Home, Dashboards, Logs / App, and a search bar. The main panel displays a list of log entries for the 'sociallenderblogDev' container, including timestamps, log levels, and messages. A search bar at the top allows filtering by 'container' and 'String Match'. The right sidebar shows a detailed view of the selected log entry, including the full log message and associated metadata.

Issue(s) Encountered

Issues Encountered

Syntax Errors in config.alloy: Initial Alloy configuration contained syntax mistakes, which prevented Alloy from starting or parsing the configuration correctly.

- All syntax errors were corrected, ensuring the configuration complied with Alloy's expected format.

Security Group / Network Connectivity Issue for Loki: The test server was unreachable to the infrastructure Loki endpoint because the infrastructure server's inbound rules was not open for that port.

- Resolved by updating the security group of the Infrastructure server to allow inbound traffic from the test server's IP address.

Exporter Authentication Errors: Redis and Valkey exporters initially failed due to authentication requirements.

- Redis was resolved by providing the correct password.
- Valkey required proper port mapping (9122) and network

Old Timestamp Limitation: Alloy was unable to scrape metrics or logs older than a certain timestamp.

- To resolve this, the query or log collection period was reduced to the last 7 days, ensuring Alloy could successfully retrieve historical data.

Recommendations

Recommendations

Configuration Management

- Validate config.alloy syntax before deployment.
- Maintain versioned configuration backups.

Log and Metrics Retention

- Configure Alloy and exporters to handle appropriate retention
- Define centralized log retention policies for long-term observability.

Network & Security

- Verify all required ports are open in security groups before deployment.

Monitoring Stack Management

- Add container health checks to detect failures early.

Wrap-up

Wrap Up

Successfully deployed a centralized monitoring stack using Grafana Alloy, Prometheus, and Loki, enabling container-level metrics and log collection. Key deployment challenges including timestamp limitations, syntax errors, and security group restrictions were resolved, establishing a reproducible and maintainable setup suitable for scaling or further integration.

This deployment provides full-stack observability and serves as a robust foundation for future monitoring across additional servers or environments.